

Spatial Mental Modeling from Limited Views

Dissertation Progress Report — Milestones 1 and 2

Reproducing ViperGPT without Codex: a Prompt-Specification Gap and a Depth-Reasoning Gap

Student Sankarshan
Supervisor Dr. Shrikant Malviya
Programme M.Tech Data Science, SVNIT Surat
Date 12 August 2026
Repository mdhumayun7/Vipergpt, tag m2-complete

Summary. ViperGPT (Surís et al., ICCV 2023) answers visual queries by prompting a code LLM to emit a Python program over vision modules. Its generator, Codex, has been retired, so every present-day reproduction must substitute an open-weights model. We rebuilt the pipeline end to end on an H100 cluster and report two results. *First*, the released prompt never states that a grounding program must return an `ImagePatch`; with Qwen2.5-Coder-7B this makes **98.8%** of programs task-invalid and yields **0.00%** accuracy on RefCOCO. Adding a return-type contract and two exemplars gives **39.20%** (RefCOCO) and **31.80%** (RefCOCO+). *Second*, having fixed that, accuracy splits sharply by the kind of spatial language: 2D image-plane queries reach 35.29%, while depth and relational queries reach **6.38%**. The exposed API cannot express depth relations and the prompt discourages geometry. That gap defines the dissertation’s contribution.

1 Pipeline and what was built

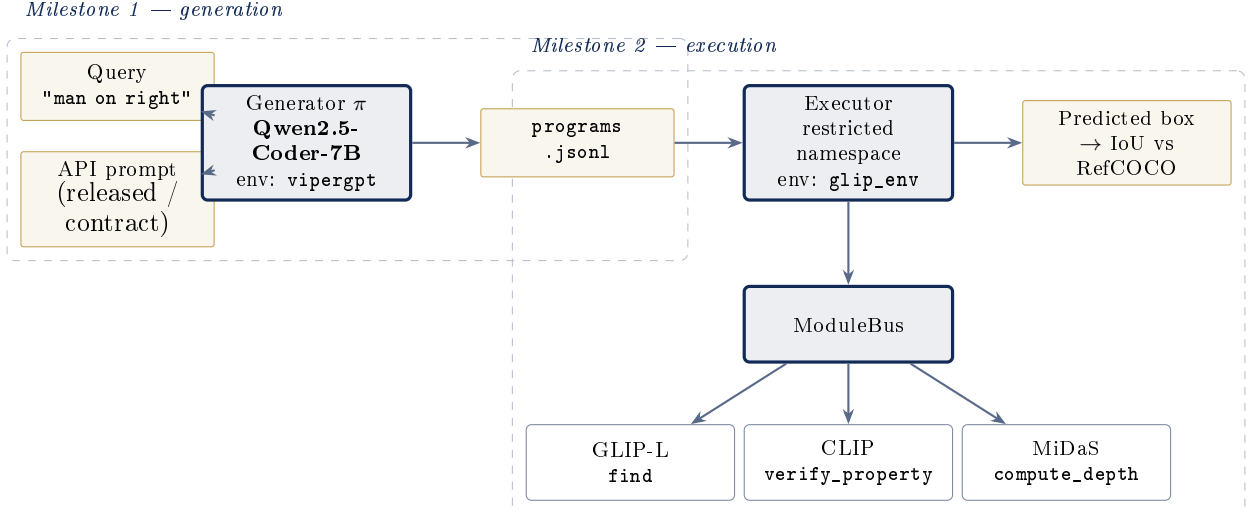


Figure 1: The reproduced pipeline. Generation and execution are split at a JSONL boundary because the GLIP fork requires torch 2.1 while the generator runs on torch 2.13 — the two cannot share an interpreter. The split also guarantees that prompt conditions are compared on *identical* programs.

Infrastructure. SVNIT HPC, 1×H100 NVL per job via SLURM `-gres=shard:N`. Building GLIP was the principal engineering risk: the official environment pins CUDA 11.6, which has no `sm_90` support, so the published setup *cannot run on this hardware at all*. It now builds against torch 2.1.2/cu121 with `sm_80;sm_90` targets after replacing removed `THC/*` headers in seven CUDA files

and NumPy 2.0 aliases in five Python files (D5, D6). Correctness was verified independently: the CUDA NMS kernel on a hand-checked case, then GLIP against RefCOCO ground truth (mean IoU 0.776 at threshold 0.2).

2 Result 1 — the released prompt omits the grounding contract

The released `api.prompt` is task-agnostic. Its in-context examples return strings (`simple_query`, `best_text_match`, `llm_query`), and nothing states that a grounding program must return an `ImagePatch`. It further instructs: “Use base Python (comparison, sorting) for basic logical operations, left/right/up/down, math, etc.”

Released prompt invalid

```
# Query: "man on right"
def execute_command(image) -> str:
    image_patch = ImagePatch(image)
    man_patches = image_patch.find("man")
    if man_patches:
        man_patch = max(man_patches,
                        key=lambda m: m.
                            right)
        return man_patch.simple_query(
            "Who is on the right?")
    return "No man found."
```

Returns a sentence. IoU undefined.

+ grounding contract valid

```
# Query: "man on right"
def execute_command(image) -> ImagePatch:
    image_patch = ImagePatch(image)
    man_patches = image_patch.find("man")
    man_patches.sort(
        key=lambda p: p.horizontal_center)
    return man_patches[-1]
```

Returns a patch. Relation resolved.

The intervention is three sentences stating the return type and forbidding dropped spatial relations, plus two worked grounding exemplars. Nothing else changed.

Table 1: Effect of the grounding contract. Identical model, decoding and data; only the prompt differs. Static columns $n = 500$ per cell; execution $n = 500$ per cell.

Metric	Released	+ Contract	
<i>Static analysis of generated programs (Milestone 1)</i>			
Task-invalid return rate, RefCOCO	98.8%	0.2%	improved
Task-invalid return rate, RefCOCO+	98.2%	0.0%	improved
Spatial constraint dropped, RefCOCO	17.3%	1.7%	improved
Parse rate, RefCOCO	99.8%	96.6%	degraded
Mean API calls per program	2.45	1.66	simpler
<i>End-to-end execution, accuracy IoU ≥ 0.5 (Milestone 2)</i>			
RefCOCO / testA	0.00%	39.20%	—
RefCOCO+ / testA	0.00%	31.80%	—
Programs returning a patch, RefCOCO	0 / 500	443 / 500	—
Programs returning a string	336 / 500	1 / 500	—

The zero is not a crash. Of 500 baseline programs on RefCOCO, 336 executed to completion and returned a *string*; none returned a patch. Grounding is scored by IoU against a box, so a sentence cannot score however correct its English. Missing modules were ruled out as a cause: `simple_query` and `llm_query` are given neutral empty-string fallbacks (D10) so that no program can fail for lack of a module we chose not to load. That concession strictly favours the baseline and does not change its score.

A second, unanticipated effect. Under the released prompt many generations never defined `execute_command` at all — the model emitted prose or malformed markdown: 136/500 (27.2%) on RefCOCO and 214/500 (42.8%) on RefCOCO+, against zero under the contract. The contract disciplines output *format* as well as return type; these are separable failures and are reported separately.

Robustness. Greedy decoding is deterministic, so variance was estimated by sampling ($T=0.7$, seeds 1–3, RefCOCO, $n=500$): 36.07 ± 1.75 against 39.20 greedy. Greedy beats all three sampled seeds, as expected when one program shape is correct.

3 Result 2 — the spatial gap, and what it is made of

With the contract in place, accuracy splits by subset (relational-term lexicon):

Table 2: Spatial breakdown, grounding-contract condition. The baseline scores 0 everywhere and is omitted.

Dataset	Subset	n	Accuracy	Mean IoU
RefCOCO	spatial	289	35.29%	0.3475
RefCOCO	non-spatial	211	44.55%	0.4378
RefCOCO+	spatial	47	6.38%	0.1037
RefCOCO+	non-spatial	453	34.44%	0.3474

We expected the gap to *shrink* on RefCOCO+, since that dataset forbids spatial relations. It widened threefold. A manual audit of all 47 RefCOCO+ spatial queries explains why: the two subsets contain **different kinds of spatial language**.

	RefCOCO	RefCOCO+
Type	2D image-plane	depth and inter-object
Examples	<i>man on right, person bottom left</i>	<i>man closest to us, farthest man, umpire behind catcher</i>
Resolvable by	sorting patch coordinates in base Python	requires <code>compute_depth</code> or 3D geometry
Accuracy	35.29%	6.38%

Forbidding spatial language removed viewer-frame *directional* expressions and left depth-ordering and relational ones. The released prompt’s instruction to use base Python for “left/right/up/down” works for the first and cannot work for the second. Consistently, Milestone 1 found `compute_depth` invoked 29 times against 547 for **find** and 5 for **distance**: the geometric API is effectively unused, by design of the prompt.

Two further observations support this reading. All three successes in the RefCOCO+ spatial subset are depth queries with very high IoU (0.97, 0.96, 0.92) — when depth ordering *is* attempted correctly, localisation is excellent; it is usually not attempted. And across seeds, spatial variance is roughly four times non-spatial variance (± 3.35 vs ± 0.72): programs handling spatial relations are not merely less accurate but less *stable*, sometimes resolving a relation by sorting and sometimes dropping it entirely.

Caveat. $n=47$, so 6.38% is 3 of 47 and the interval is wide. The lexicon also produces roughly five false positives in that subset, which would push true depth-query accuracy lower still. The direction is informative; the value is not.

4 Reproducibility record

ID	Deviation	Reason / effect
D1	Codex $\rightarrow$ Qwen2.5-Coder-7B	Codex retired; no faithful option. The substitution is what exposes Result 1, and is the dominant term in the gap to 72.0.
D2	Scope: grounding only	GQA/OK-VQA/NExT-QA need $\sim$ 250 GB against 231 GB quota; OK-VQA also needs GPT-3, likewise retired.
D3	RefCOCO annotations from Internet Archive	The canonical UNC host no longer resolves. Archived zips are byte-identical. No effect.
D5, D6	GLIP built for sm_90	CUDA 11.6 has no sm_90. Headers and NumPy aliases updated; kernels unchanged. Verified numerically.
D8	GLIP threshold 0.2 (paper unspecified)	Swept on 20 samples: 0.5 loses 8/20 detections, 0.2 gives mean IoU 0.776. Material and under-validated.
D9	X-VLM $\rightarrow$ CLIP	X-VLM weights are Google-Drive-only. CLIP scores globally, so attribute checks are weaker — costliest on RefCOCO+.
D10	<code>simple_query/llm_query</code> return ""	Loading BLIP-2-XXL and an 8B text LLM is out of scope. Favours the baseline; no effect on the contract condition.

Two recorded failures. A first hypothesis — that spatial queries would collapse into a single opaque `simple_query` call, as the paper’s GQA prompt instructs — was tested and rejected (collapse rate $\approx$ 0%); that instruction applies to GQA, and grounding cannot use `simple_query` because it must return a patch. Separately, an early 500-sample execution ran on a login node with no GPU, producing a complete and entirely fake results table; those runs were deleted and job scripts now assert `torch.cuda.is_available()`. Silent CPU fallback is the most dangerous failure mode encountered, because it yields plausible output rather than an error.

Artefacts. Every run writes a timestamped directory with resolved config, per-query programs, per-sample records and a summary; SLURM job IDs are recorded; `docs/research_log.md` (540+ lines) documents the full arc including failures.

5 Where the gap to 72.0 comes from

Reproduced 39.20 against a reported 72.0. Raw GLIP top-box accuracy on this data is 78.9%, so perception caps the system near 79% before any program runs; program-level selection loses roughly 40 points from there. Ordered by likely size, the contributors are D1 (generator substitution, dominant), the perception ceiling, D9 (X-VLM $\rightarrow$ CLIP, costliest on RefCOCO+, consistent with 31.80 vs 39.20), and D8 (threshold tuned on 20 samples). No claim of a faithful reproduction is made: what is reproduced is the method; what is measured is what happens to it when its generator is replaced.

6 Proposed contribution

The measured problem is specific: *the API cannot express depth relations, and the prompt discourages geometry*. `compute_depth()` returns one scalar — the median relative depth of a patch — from which “A is behind B” is expressible but “the third-closest person”, “between A and B in depth”, and metric size are not.

Proposal. Replace the scalar with a back-projected representation and expose operations over it: `position_3d`, `depth_order`, `is_behind`, `distance_3d`, `is_between_3d`, `physical_size`; swap MiDaS for a metric depth model; and remove the anti-geometry instruction from the prompt. `depth_order` is the highest-value single primitive — *closest*, *nearest*, *farthest* account for most of the audited subset.

Evaluation reuses the existing pipeline unchanged. Three conditions: released prompt (0.00%), grounding contract (39.20% / 6.38% on depth queries), and the depth-grounded API. The primary claim is improvement on the depth subset; the necessary control is that the 2D subset and non-spatial sets do not degrade. The paper’s own interventional protocol (Sec. 5.2) applies: neutralising **depth_order** must cause a measurable drop, or the primitive is decorative. A prompt-only ablation separates *encouraging* geometry from *providing* it.

Honest risk. The generator may ignore the new API as it ignores `compute_depth` today. Call frequency will therefore be measured directly, as in Milestone 1. If the API is not invoked, the finding concerns prompt adherence rather than geometry — still reportable, but weaker.

Metric depth is claimed only as *ordering*: COCO has no camera intrinsics, so absolute metres would rest on an assumed field of view. **depth_order** and **is_behind** require only monotonicity, which relative depth already provides.

7 Limitations and next steps

1. One generator (Qwen2.5-Coder-7B). Whether the prompt effect is scale-dependent is untested; a 1.5B/32B sweep needs no new infrastructure.
2. GLIP threshold tuned on 20 samples (D8); to be re-swept before final numbers.
3. X-VLM substituted by CLIP (D9); the substitution most likely to be costing accuracy, especially on RefCOCO+.
4. Seed variance measured on RefCOCO only, and by sampling rather than by retraining — the standard caveat for a decoding-only estimate.
5. Spatial subsets are identified by lexicon, not human annotation.

All figures traceable to a run directory, an archived summary in `results/m2_summaries/`, and a SLURM job ID. Milestone 1: jobs 29187, 29193. Milestone 2: jobs 29290, 29292, 29301, 29306. Repository tags `m1-complete`, `m2-glip-verified`, `m2-execution`, `m2-complete`.